

LC 1857: Largest Color Value in a Directed Graph

Authored by: Arpit Raj, LNMIIT Jaipur | Date: July 31, 2026

Problem Breakdown

- **Objective:** Find the max most frequent no. of colored nodes having some colour in a single path.
- **Constraint:** The colour is in lowercase english letters.
- **Challenge:** Trying every path is expensive.

Example Path: 0(a) → 2(a) → 3(c) → 4(a)

The path has **a: 3** and **c: 1**.
Hence, **dp = 3**.

Key Intuition & Approach

basic aadya ye he ki
ek dp table bana lenge jisme we can store
ki iss falaane node me falaana colour aane ki max freq kya he

- So we can make use of DP state which will store best color frequency till a node.
- **$dp[node][26]$** → max count of color **C** on any path ending at node.
- **Why Kahn's Algorithm?**
kahn bhi use krna padega kyunki har ek parent jab process ho jaaye tabhi child ka state correctly update ho skega.
NEED TO USE KAHN as a node should be processed only after all parent not.

State Transition

Let $u \rightarrow v$.

If v has color a , then:

$$dp[v][a] = \max(dp[v][a], dp[u][a] + 1)$$

and for other colors:

$$dp[v][c] = \max(dp[v][c], dp[u][c])$$

C++ Implementation

```

class Solution {
public:
    int largestPathValue(string colors, vector<vector<int>>& edges) {

        // basic aadya ye he ki
        // ek dp table bana lenge jisme we can store
        // ki iss falaane node me falaana colour aane ki max freq kya he
        // dp[node][26]
        // kahn bhi use krna padega kyunki har ek parent jab process ho jaaye
        // tabhi child ka state correctly update ho skega

        int n = colors.size();
        vector<vector<int>> adj(n);
        vector<int> indeg(n, 0);
        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];

            adj[u].push_back(v);
            indeg[v]++;
        }

        vector<vector<int>> dp(n, vector<int>(26, 0));
        queue<int> q;
        for(int i = 0; i < n; i++) {
            if(indeg[i] == 0) {
                q.push(i);
                dp[i][colors[i]-'a'] = 1;
            }
        }

        int ans = 0;
        int process = 0; // given he question me cycle exist kr skti isliye
                        // track rkhenge ki saare node process hue ya nhi

        while(!q.empty()) {
            int curr = q.front();
            q.pop();
            process++;

            for(int col = 0; col < 26; col++) {
                ans = max(ans, dp[curr][col]);
            }

            for(auto &it : adj[curr]) {
                // har ek colour apne parent se chhap lenge
                // make sure krenge ki max freq colour inherit kre
                // eg: one parentA can be rich in red , one parentB can be rich in blue
                // to agar blue inherit krna ho to parentB se lenge
            }
        }
    }
};

```

```

        for(int col = 0; col < 26; col++) {
            dp[it][col] = max(dp[it][col], dp[curr][col]);
        }

        indeg[it]--;
        if(indeg[it] == 0) {
            dp[it][colors[it]-'a']++; // khud ka contri
            q.push(it);
        }
    }

    if(process == n) return ans;
    else return -1;
}

};

```